

Tema 1

Pregunta 1 (Definir el concepto de lenguaje de proposiciones). Si σ un conjunto finito de átomos, definimos el lenguaje de las σ -fórmulas proposicionales como el conjunto de elementos generados por las siguientes reglas:

1. Todo elemento de σ es una fórmula.
2. Si φ es una fórmula, $\neg\varphi$ también lo es.
3. Si φ, ψ son fórmulas, entonces $(\varphi \vee \psi)$, $(\varphi \wedge \psi)$, $(\varphi \rightarrow \psi)$ y $(\varphi \leftrightarrow \psi)$ son también fórmulas.

Pregunta 2 (Definir el concepto de interpretación y los conceptos de tautología, fórmula satisfactible y contradicción).

Si σ un conjunto finito de átomos, una σ -interpretación es una asignación de los valores de verdad a los elementos de σ .

1. Una fórmula es una tautología, si es cierta en todas las interpretaciones.
2. Una fórmula es satisfactible, si es cierta en alguna interpretación.
3. Una fórmula es una contradicción, si es falsa en todas las interpretaciones.

Pregunta 3 (Definir el concepto de fórmulas equivalentes). Dos fórmulas φ, ψ son lógicamente equivalentes, si para toda interpretación I , tenemos que $I(\varphi) = I(\psi)$. Es decir, dos fórmulas son lógicamente equivalentes, si no hay ninguna interpretación que las distinga.

Pregunta 4 (Definir los conceptos de literal, cláusula y fórmula en forma normal conjuntiva). Un literal es un átomo o a la negación de un átomo.

Una cláusula es una disyunción de literales.

Una fórmula φ está en forma normal conjuntiva, si φ es de la forma $\varphi_1 \wedge \dots \wedge \varphi_n$ donde $\varphi_1, \dots, \varphi_n$ son cláusulas.

Pregunta 5 (Explicar el algoritmo visto en clase para transformar una fórmula proposicional en una fórmula equivalente en forma normal conjuntiva). Hay que seguir el siguiente proceso:

1. Aplicar las reglas:

$$\varphi \leftrightarrow \psi \equiv (\varphi \rightarrow \psi) \wedge (\psi \rightarrow \varphi)$$

$$\varphi \rightarrow \psi \equiv \neg\varphi \vee \psi$$

2. Aplicar las reglas:

$$\neg(\neg\varphi) \equiv \varphi$$

$$\neg(\varphi \vee \psi) \equiv \neg\varphi \wedge \neg\psi$$

$$\neg(\varphi \wedge \psi) \equiv \neg\varphi \vee \neg\psi$$

3. Aplicar las reglas:

$$\varphi \vee (\psi \wedge \chi) \equiv (\varphi \vee \psi) \wedge (\varphi \vee \chi)$$

$$\varphi \wedge (\psi \vee \chi) \equiv (\varphi \wedge \psi) \vee (\varphi \wedge \chi)$$

Pregunta 6 (Explicar en qué consiste el problema SAT y explicar el interés de dicho problema). El problema SAT consiste en determinar si una fórmula proposicional en forma normal conjuntiva es satisfactible.

El interés del problema SAT radica en que hay muchos problemas prácticos que se pueden representar mediante una fórmula proposicional, de manera que el hecho de que el problema tenga solución significa que la fórmula asociada es satisfactible.

Pregunta 7 (Explicar qué es un SAT-solver y dar ejemplos de problemas que se pueden resolver mediante SAT-solvers). Un SAT-solver es un programa que resuelve el problema SAT.

Entre los problemas que se pueden resolver mediante SAT-solvers, tenemos los siguientes:

1. El problema de colorear un mapa de un continente con cuatro colores de manera que no haya dos países vecinos que tengan el mismo color.
2. El problema de instalar paquetes de actualización en un ordenador satisfaciendo una serie de restricciones.
3. El problema de formar un equipo de personas lo más pequeño posible para realizar una serie de tareas.
4. Problemas para confeccionar horarios de hospitales, escuelas o líneas aéreas.
5. La resolución de sudokus.

Pregunta 8 (Explicar el interés del método de Davis-Putnam). Es el método en el que está basado el diseño de los SAT-solvers.

Es un método mucho más eficiente que el algoritmo consistente en generar todas las interpretaciones posibles y ver si alguna de ellas satisface la fórmula de la entrada.

El interés del método también radica en el hecho de que las reglas de Davis-Putnam preservan la satisfactibilidad de la fórmula de la entrada.

Pregunta 9 (Definir el concepto de resolvente de dos fórmulas proposicionales). Para dos cláusulas proposicionales φ_1, φ_2 , si existe un literal ψ_1 en φ_1 que es complementario de un literal ψ_2 en φ_2 , se suprimen ψ_1, ψ_2 de φ_1, φ_2 respectivamente, y se construye la disyunción de las cláusulas resultantes. La cláusula así construida se llama resolvente de φ_1 y φ_2 .

Pregunta 10 (Explicar en qué consiste el algoritmo de resolución para lenguajes de proposiciones). El algoritmo está basado en el teorema de resolución y se utiliza para diseñar demostradores. Recibe como entrada una lista de fórmulas $\varphi_1, \dots, \varphi_n, \varphi$, y da como salida "éxito" si φ es consecuencia lógica de $\varphi_1, \dots, \varphi_n$ y da "fallo" en caso contrario.

El algoritmo consta de las siguientes etapas:

1. Se calcula una forma normal conjuntiva $\psi_1 \wedge \dots \wedge \psi_k$ de la fórmula $\varphi_1 \wedge \dots \wedge \varphi_n \wedge \neg\varphi$
2. El algoritmo va calculando resolventes a partir de las fórmulas $\psi_1, \dots, \psi_k$ hasta que se obtenga la cláusula vacía $\square$ o hasta que no se puedan computar más resolventes.

El algoritmo dará entonces "éxito" si se obtiene la cláusula vacía, y dará "fallo" en caso contrario.

Tema 2

Pregunta 11 (Definir el concepto de predicado y dar ejemplos de predicados). Un predicado es una expresión formal, cuyo tipo de datos es booleano.

Las siguientes expresiones son ejemplos de predicados:

1. $x + y = 2$.
2. $x + y \leq z$.
3. Existe un entero positivo $n < 100$ tal que para todo entero $m \in \{1, \dots, k\}$, $A[m] < n$.

Pregunta 12 (Explicar en qué se diferencian los términos de las fórmulas). En los términos aparecen variables, símbolos de constante y símbolos de operador, pero no pueden aparecer símbolos de predicado.

En las fórmulas, aparecen siempre términos y símbolos de predicado.

En una interpretación, el resultado de evaluar un término es un elemento del dominio de la interpretación, mientras que el resultado de evaluar una fórmula es un valor booleano.

Pregunta 13 (Definir los conceptos de término y átomo en lenguajes de predicados). El conjunto de los términos es el conjunto de elementos generado por las siguientes reglas:

1. Toda variable es un término.
2. Todo símbolo de constante es un término.
3. Si f es un símbolo de función de n argumentos y $t_1, \dots, t_n$ son términos, entonces $f(t_1, \dots, t_n)$ es un término.

Un átomo es una expresión de la forma $Rt_1 \dots t_n$ donde R es un símbolo de predicado de n argumentos y $t_1, \dots, t_n$ son términos.

Pregunta 14 (Definir el concepto de fórmula de un lenguaje de predicados). El conjunto de las fórmulas es el conjunto de elementos generado por las siguientes reglas:

1. Todo átomo es una fórmula.
2. Si φ es una fórmula, $\neg\varphi$ también lo es.
3. Si φ, ψ son fórmulas, entonces $(\varphi \vee \psi)$, $(\varphi \wedge \psi)$, $(\varphi \rightarrow \psi)$ y $(\varphi \leftrightarrow \psi)$ son también fórmulas.
4. Si φ es una fórmula y x es una variable, entonces $\exists x\varphi$, $\forall x\varphi$ son fórmulas.

Pregunta 15 (Definir los conceptos de variable libre y variable ligada en una fórmula, y dar ejemplos de variables libres y ligadas en una fórmula). Una variable x es libre en una fórmula φ , si hay alguna aparición de la variable x en φ que no está afectada por ningún cuantificador.

Una variable x está ligada en una fórmula φ , si todas las apariciones de la variable x en φ están afectadas por cuantificadores.

Por ejemplo, en la fórmula $\exists x(Rxy \wedge \forall y(Py \vee Ryz))$ las variables y, z son libres y la variable x es ligada.

Pregunta 16 (Definir el concepto de consecuencia lógica y el concepto de demostrador). Una fórmula φ es consecuencia lógica de fórmulas $\varphi_1, \dots, \varphi_n$, si la fórmula $(\varphi_1 \wedge \dots \wedge \varphi_n) \rightarrow \varphi$ es una tautología.

Un demostrador es un programa que recibe como entrada una lista de fórmulas $\varphi_1, \dots, \varphi_n, \varphi$, y entonces determina si φ es consecuencia lógica de $\varphi_1, \dots, \varphi_n$.

Pregunta 17 (Explicar el interés de los demostradores). Los demostradores tienen interés especialmente en la verificación de sistemas informáticos. En el caso del diseño de hardware, se utilizan para verificar que los circuitos que realizan las operaciones matemáticas de la CPU de un ordenador están correctamente diseñados.

Los demostradores tienen también interés en el campo de la programación declarativa, ya que los interpretadores de diversos lenguajes de programación, como es el caso del lenguaje Prolog, son demostradores de teoremas.

Pregunta 18 (Explicar en que consiste la regla de resolución para lenguajes de predicados). Las entradas de la regla son dos cláusulas $\varphi_1 \vee \psi_1 \vee \varphi'_1$ y $\varphi_2 \vee \neg\psi_2 \vee \varphi'_2$ que no comparten variables y tales que ψ_1 y ψ_2 son átomos unificables.

La salida de la regla es la fórmula $(\varphi_1 \vee \varphi'_1 \vee \varphi_2 \vee \varphi'_2)\lambda$ donde λ es el unificador de ψ_1 y ψ_2 obtenido mediante el algoritmo de unificación.

Pregunta 19 (Explicar en que consiste el método de resolución para los lenguajes de predicados). Las entradas son fórmulas $\varphi_1, \dots, \varphi_n, \varphi$ de un lenguaje de predicados. El método consiste en las siguientes dos etapas.

1. Calcular formas clausales $(\varphi_1)^{cl}, \dots, (\varphi_n)^{cl}, (\neg\varphi)^{cl}$ de las fórmulas $\varphi_1, \dots, \varphi_n, \neg\varphi$ respectivamente, y tomar las cláusulas $\psi_1, \dots, \psi_m$ que aparecen en los núcleos de dichas formas clausales.
2. Calcular resolventes a partir de las entradas $\psi_1, \dots, \psi_m$. Si en algún momento se obtiene la cláusula vacía $\square$, el algoritmo para y devuelve "éxito".

Pregunta 20 (Explicar el interés del método de resolución). Es el método en el que está basado el interpretador del lenguaje de programación Prolog, y se utiliza también en el diseño de los interpretadores de otros lenguajes de programación.

Es un método que se puede utilizar en el diseño de demostradores que verifiquen la corrección de sistemas informáticos.

Se utiliza también para diseñar demostradores de teoremas matemáticos por ordenador, como es el caso de los demostradores Otter y Prover9.